



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina CACHOPO

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio CACHOPO
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
Inyección de comandos vía parámetro codificado en Base64 → RCE (Crítica)	7
Hashes SHA1 con sal débiles y expuestos → contraseña de root (Alta)	12
Historial de Cambios	14
Aviso Legal	14



Resumen Ejecutivo

Se ha comprometido la máquina **CACHOPO** de DockerLabs obteniendo **ejecución remota de código** y, tras crackear los hashes del sistema, la **contraseña de root**. El panel de reservas trata la entrada como un valor **Base64**: los mensajes de error revelan que decodifica el parámetro, y colando un **comando codificado en Base64** se consigue **inyección de comandos** y una shell en el servidor.

Ya dentro, en `/com/perosnal/hash.lst` se encuentran varios **hashes SHA1 con sal**. Con la herramienta `SHA_Decrypt` y el diccionario `rockyou.txt` se **crackea** el hash correspondiente, obteniendo la **contraseña de root** (con la que se completaría la escalada a superusuario).

El impacto es **alto/crítico**: una deserialización/decodificación insegura de la entrada deriva en RCE, y el uso de **hashing débil** (SHA1 con sal, sin *key stretching*) permite recuperar la contraseña de root.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **CACHOPO** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: El panel de reservas procesa un valor **codificado en Base64**; colando un comando codificado se logra **inyección de comandos (RCE)**. Dentro, en `/com/perosnal/hash.lst` hay hashes **SHA1 con sal** que, crackeados con diccionario, revelan la **contraseña de root**.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **2** vulnerabilidades: **1 Crítica(s)**, **1 Alta(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

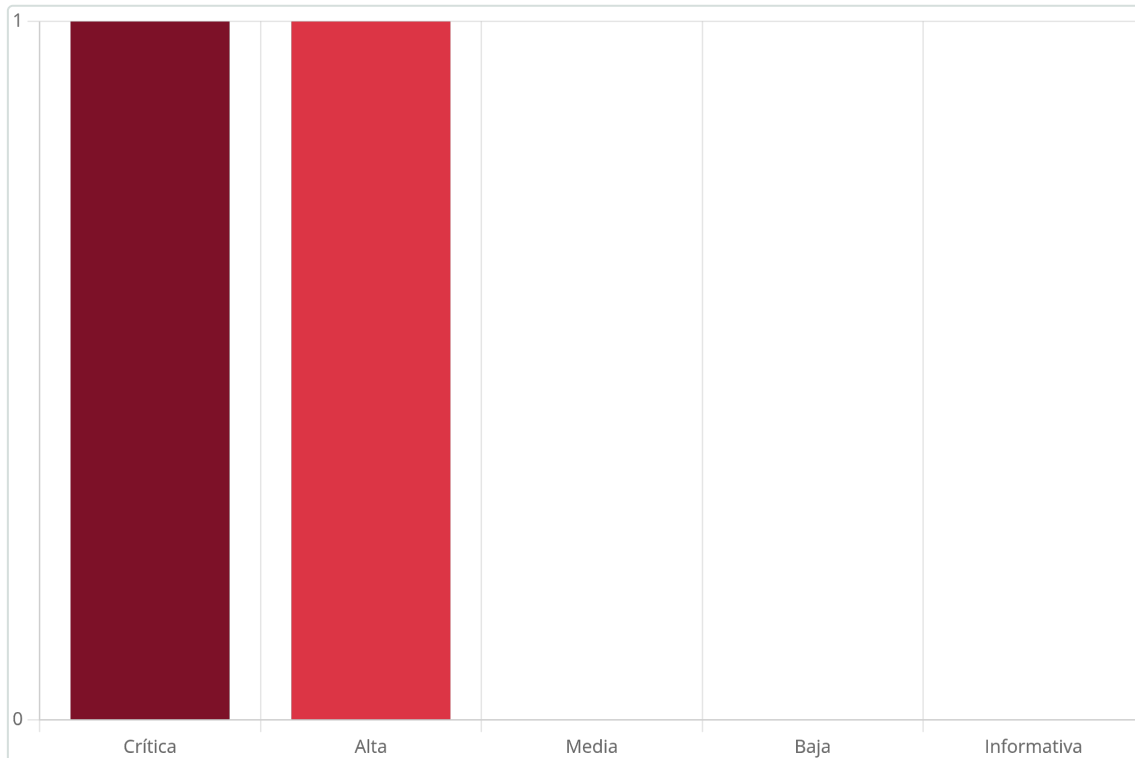


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
Inyección de comandos vía parámetro codificado en Base64 → RCE	Crítica
Hashes SHA1 con sal débiles y expuestos → contraseña de root	Alta

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. Inyección de comandos vía parámetro codificado en Base64 → RCE (Crítica: 9.8)

Afecta a: Servicio web (puerto 80) — panel de reservas (parámetro en Base64)

El panel de reservas **decodifica en Base64** el valor recibido y lo procesa de forma insegura. Enviando un **comando codificado en Base64** se logra **inyección de comandos** y una shell en el servidor.

2. Hashes SHA1 con sal débiles y expuestos → contraseña de root (Alta: 7.8)

Afecta a:

- `/com/perosnal/hash.lst` (hashes SHA1 con sal accesibles)
- Cuenta `root` (contraseña recuperable por diccionario)



El fichero `/com/perosnal/hash.lst` almacena hashes **SHA1 con sal**, accesibles y crackeables por diccionario. Con `SHA_Decrypt` y `rockyou.txt` se recupera la **contraseña de root**.



Detalle de Vulnerabilidades

1. Inyección de comandos vía parámetro codificado en Base64 → RCE

Crítica Puntuación CVSS: 9.8

Afecta a: Servicio web (puerto 80) — panel de reservas (parámetro en Base64)

Recomendación rápida: Validar y tipar la entrada tras decodificar; nunca pasar datos del usuario a un intérprete/shell.

Resumen

El panel de reservas **decodifica en Base64** el valor recibido y lo procesa de forma insegura. Enviando un **comando codificado en Base64** se logra **inyección de comandos** y una shell en el servidor.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Al interceptar la reserva se observa que los mensajes de error cambian según se envíe un número, texto o Base64: el backend **decodifica el parámetro en Base64** y lo procesa de forma insegura. Codificando un **comando** en Base64 e inyectándolo, el servidor lo ejecuta (**inyección de comandos**), devolviendo una shell.

A continuación, paso a paso:

```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64    OpenSSH 9.6p1 Ubuntu 3ubuntu13.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 7b:98:d4:e7:ec:50:0b:b2:3a:21:76:2c:45:95:23:61 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBEjW70Fr3vBvcqYorxu8cbiA
5d1U=
|   256 5d:15:2b:28:ec:67:7e:78:3c:16:12:65:2f:59:d4:88 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKiE4JB9YEGU2mtWJP7VMmr5R60+RXwvThaYJ//r0T9h
80/tcp    open  http      syn-ack ttl 64    Werkzeug/3.0.3 Python/3.12.3
| http-methods:
|_ Supported Methods: GET HEAD OPTIONS
|_ http-title: Cahopos4-4ll
|_ http-server-header: Werkzeug/3.0.3 Python/3.12.3
| fingerprint-strings:
|_ GetRequest:
|_   HTTP/1.1 200 OK
|_   Server: Werkzeug/3.0.3 Python/3.12.3
|_   Date: Tue, 30 Jul 2024 10:51:33 GMT
|_   Content-Type: text/html; charset=utf-8
|_   Content-Length: 9332
|_   Connection: close
|_   <!DOCTYPE html>
```

Figura 2 – Hacemos el escaneo de nmap

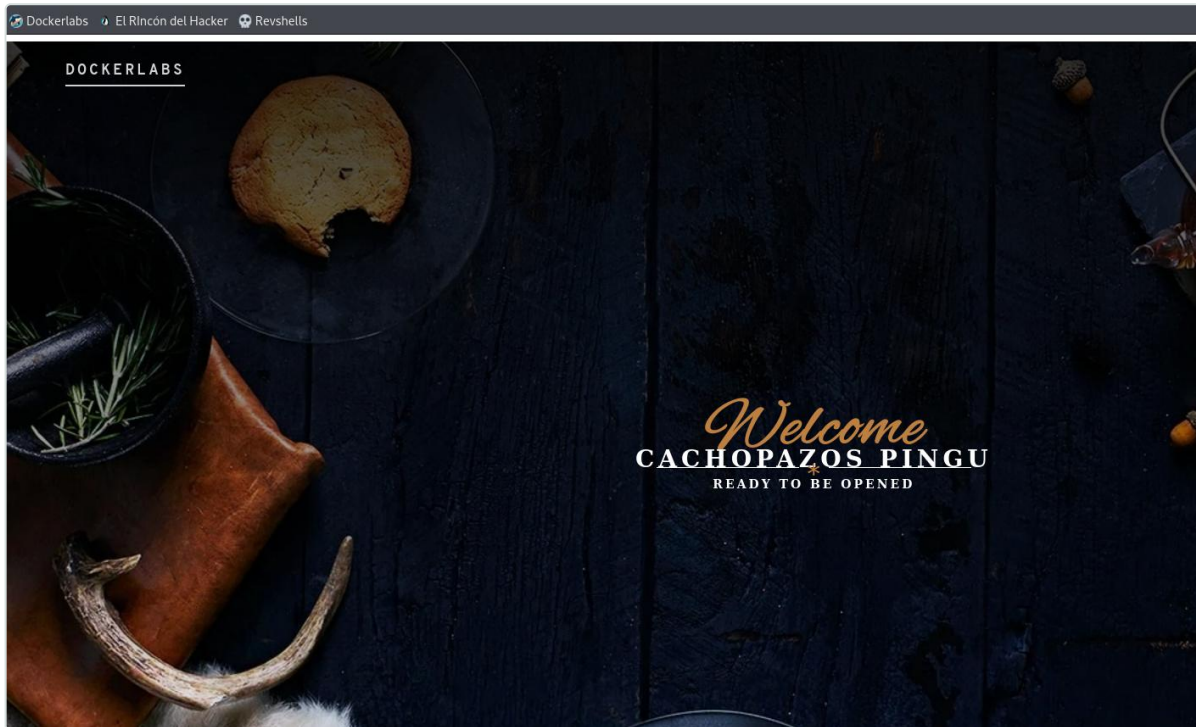


Figura 3 – Y así es la web



Make a Reservation



Submit Reservation

Figura 4 – Vemos un panel de hacer reservas



The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is active, displaying a POST request to `/submitTemplate` with a body of `userInput=123`. The 'Response' tab shows an error: 'Error: Incorrect padding'. A red arrow points to the `userInput=123` in the request body.

Figura 5 – Interceptamos la petición, pero vemos que dependiendo de si enviamos un número o un texto vemos un error diferente

The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is active, displaying a POST request to `/submitTemplate` with a body of `userInput=test`. The 'Response' tab shows an error: 'Error: 'utf-8' codec can't decode byte 0xb5 in position 0: invalid start byte'.

The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is active, displaying a POST request to `/submitTemplate` with a body of `userInput=46757`. The 'Response' tab shows an error: 'Error: Invalid base64-encoded string: number of data characters (5) cannot be 1 more than a multiple of 4'. A red arrow points to the `userInput=46757` in the request body.

Figura 6 – Si probamos solo con números nos dice algo de base64

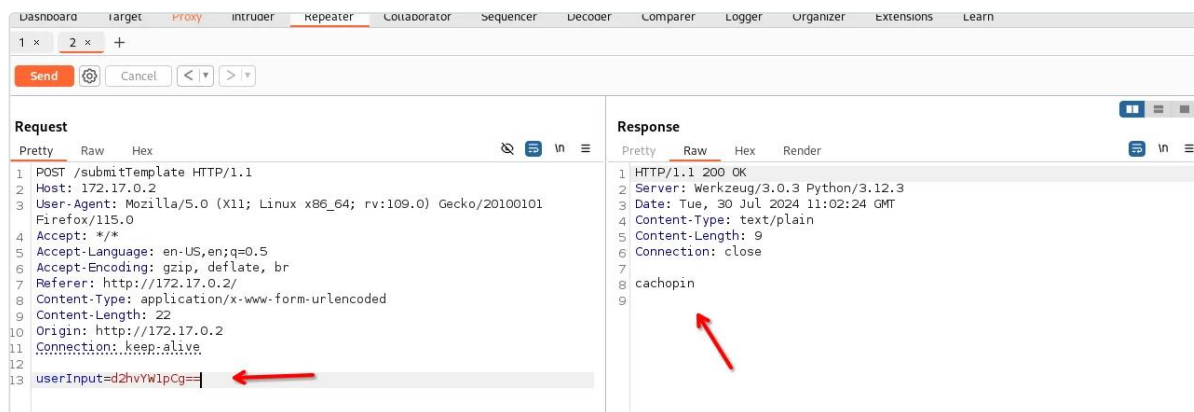


Figura 7 – Por lo que intentamos colar un comando en base64 y vemos que funciona

Recomendación

- **Validar y tipar** estrictamente la entrada **después** de decodificar (no confiar en que venga en Base64).
- **Nunca** construir comandos del sistema con datos del usuario; usar APIs parametrizadas y evitar `shell`.

Referencias

- CWE-78: OS Command Injection
- https://owasp.org/www-community/attacks/Command_Injection



2. Hashes SHA1 con sal débiles y expuestos → contraseña de root

Alta Puntuación CVSS: **7.8**

Afecta a:

- `/com/perosnal/hash.lst` (hashes SHA1 con sal accesibles)
- Cuenta `root` (contraseña recuperable por diccionario)

Recomendación rápida: No exponer los hashes; usar algoritmos lentos y salados (bcrypt/argon2); contraseñas robustas.

Resumen

El fichero `/com/perosnal/hash.lst` almacena hashes **SHA1 con sal**, accesibles y crackeables por diccionario. Con `SHA_Decrypt` y `rockyou.txt` se recupera la **contraseña de root**.

Descripción Técnica y Evidencias

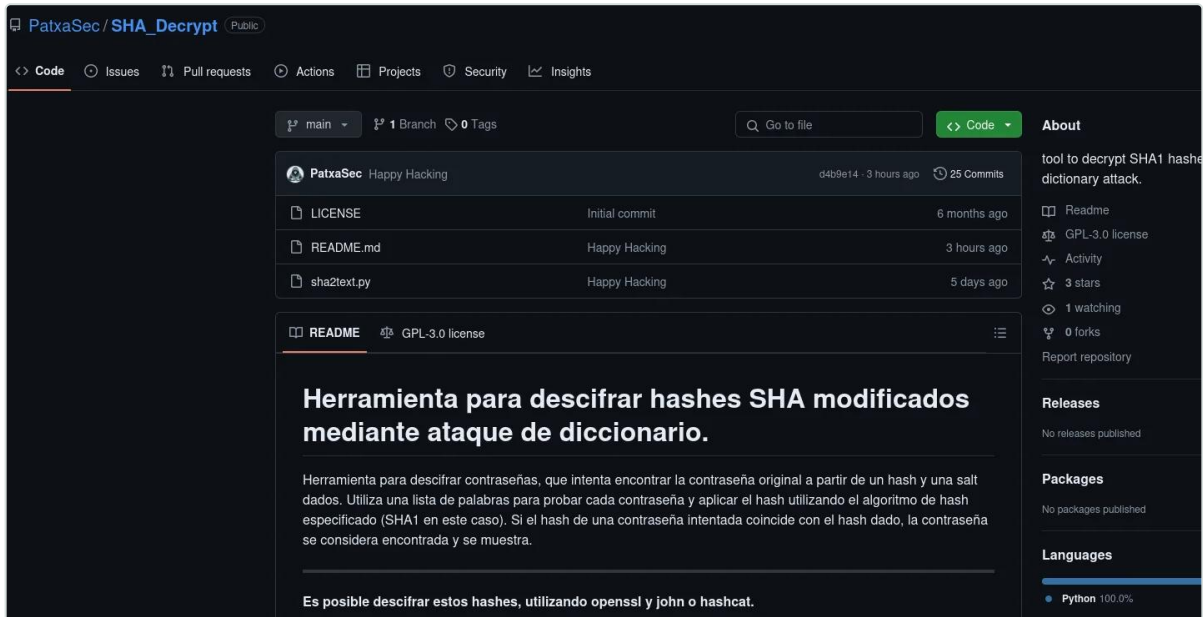
¿Qué es y por qué ocurre? Dentro del sistema, en `/com/perosnal/hash.lst` hay varios **hashes SHA1 con sal**. Al ser SHA1 un algoritmo **rápido y sin key stretching**, resulta vulnerable a ataques de diccionario: con la herramienta `SHA_Decrypt` (indicándole la sal) y `rockyou.txt` se **crackea** el hash y se obtiene la **contraseña de root** en texto plano.

A continuación, paso a paso:

```
cachopin@249c9839d143:~/app/com$ cd personal/
cachopin@249c9839d143:~/app/com/personal$ ls
hash.lst
cachopin@249c9839d143:~/app/com/personal$ cat hash.lst
$SHA1$d$GkLrWsB7LfJz1tqHBiPzuvM5yFb=
$SHA1$d$BjkVArB9RcGUs3sgVKyAvxzH0eA=
$SHA1$d$NxJmRtB6LpHs9vJYpQkErzU8wAv=
$SHA1$d$BvKpTbC5LcJs4gRzQfLmHxM7yEs=
$SHA1$d$LxVnWkB8JdGq2rH0UjPzKvT5wM1=
cachopin@249c9839d143:~/app/com/personal$
```

Figura 8 – Una vez dentro, vemos que en `/com/perosnal/hash.lst` tenemos distintos hashes

Para crackear estos hashes, usaremos la siguiente herramienta: https://github.com/PatxaSec/SHA_Decrypt



Con esta herramienta la ejecutamos de la siguiente forma:

```
python3 script.py 'd' '$SHA1$d$BjkVArB9RcGUs3sgVKyAvxzH0eA=' '/usr/share/wordlists/rockyou.txt'
```

Donde en este caso le pasamos el salto d, el cual en el contexto de hashes, especialmente cuando se trata de hashes de contraseñas, "salto" se refiere a un valor aleatorio que se utiliza para hacer que cada hash sea único, incluso si la misma contraseña se utiliza en diferentes ocasiones. Este valor ayuda a proteger contra ataques de diccionario y ataques de fuerza bruta, porque hace que el hash de una contraseña específica sea diferente cada vez que se genera.

```
(kali@kali) ~/Desktop
$ python3 script.py 'd' '$SHA1$d$BjkVArB9RcGUs3sgVKyAvxzH0eA=' '/usr/share/wordlists/rockyou.txt'
Processing: 7% [██████████] | 992816/14344392 [00:03:00:48, 277125.93it/s]
[+] Pwnd !!! $SHA1$d$BjkVArB9RcGUs3sgVKyAvxzH0eA=:::cec1na
```

Figura 9 – Y nos habrá encontrado la contraseña de root

Recomendación

- **No almacenar/exponer** ficheros de hashes accesibles por servicios o usuarios sin privilegios.
- Usar funciones de derivación **lentas y saladas** (`bcrypt`, `scrypt`, `argon2`), no SHA1.
- Política de **contraseñas robustas**.

Referencias

- CWE-916: Use of Password Hash With Insufficient Computational Effort
- https://github.com/PatxaSec/SHA_Decrypt



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.